PRAKTIKUM 1
POINTER

NAMA: Ihsanta Zaki Sanjaya

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Dr Bima Sena Bayu Dewantara S.ST, M.T

MATA KULIAH: Praktikum Pemrograman Dasar 2

TANGGAL: Rabu, 6 Maret 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

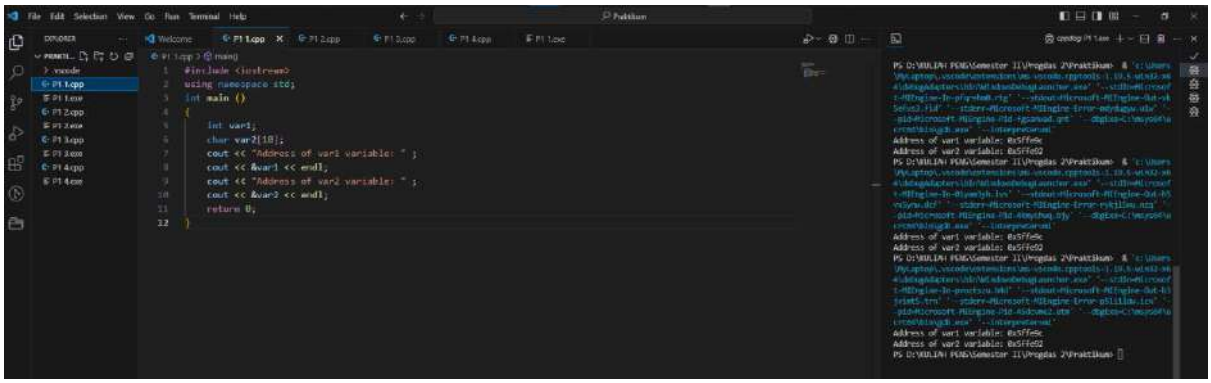
Praktikum 1
Pointer

Untuk memberikan pemahaman kepada anda tentang konsep dasar pointer dan cara menggunakan dalam program C++, maka cobalah beberapa modul percobaan dibawah ini.

1. Percobaan 1 – Konsep Dasar Pointer 1 – Alamat dari variabel

```
#include <iostream>
using namespace std;
 int main
()
{     int var1;     char var2[10];
cout << "Address of var1 variable: " ;
cout << &var1 << endl;     cout <<
"Address of var2 variable: " ;     cout <<
&var2 << endl;     return 0;
}
```

Hasil:



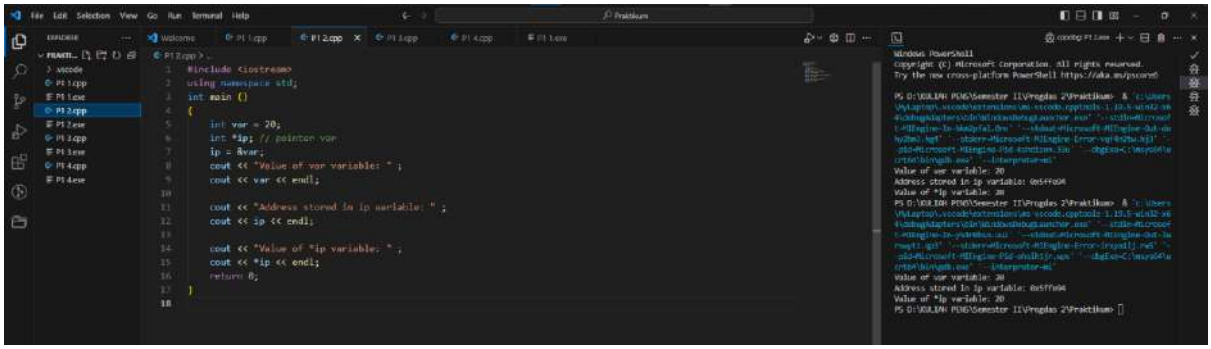
Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa alamat memori pada variable ‘var1’ dan array ‘var2’ berbeda. Alamat memori adalah lokasi fisik di mana variabel disimpan dalam memori computer, dapat berbeda tergantung pada bagaimana memorinya disusun oleh sistem saat runtime terjadi. Alamat tersebut dicetak dengan operator ‘&’ yang bisa mengembalikan alamat dari variable tersebut.

2. Percobaan 2 – Konsep Dasar Pointer 2 – Menggunakan pointer di C++

```
#include <iostream>
using namespace std;

int main ()
{     int var = 20;  // actual variable
declaration.
    int *ip;  // pointer variable     ip = &var;  // store
address of var in pointer variable     cout << "Value of
var variable: " ;     cout << var << endl;
    // print the address stored in ip pointer
variable     cout << "Address stored in ip variable:
" ;     cout << ip << endl;
    // access the value at the address available in
pointer     cout << "Value of *ip variable: " ;     cout
<< *ip << endl;     return 0;
}
```

Hasil:



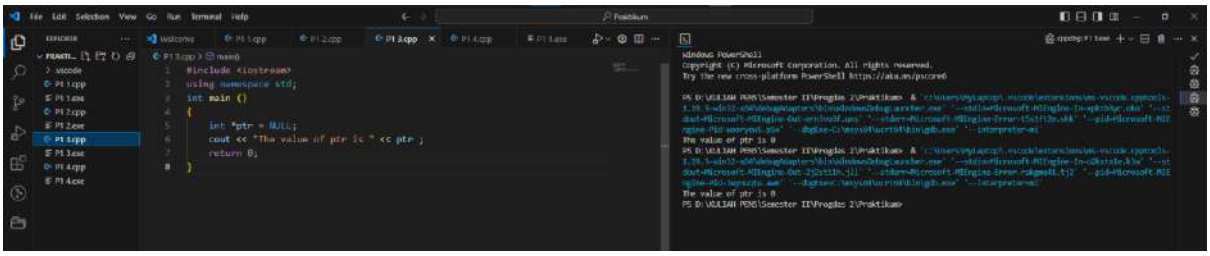
Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa dengan kita bisa menggunakan pointer untuk mengakses nilai variable dan alamatnya. Mula-mula variable ‘var’ memiliki nilai awal 20. Ketika pointer ‘ip’ menunjuk ke alamat memori ‘var’, nilai yang ditunjuk oleh ‘ip’ akan bernilai sama pada nilai dari variable ‘var’, yaitu 20.

3. Percobaan 3 – Konsep Dasar Pointer 3 – Null Pointer

```
#include <iostream>
using namespace std;

int main ()
{
    int *ptr = NULL;
    cout << "The value of ptr is " << ptr ;
return 0;
}
```

Hasil:



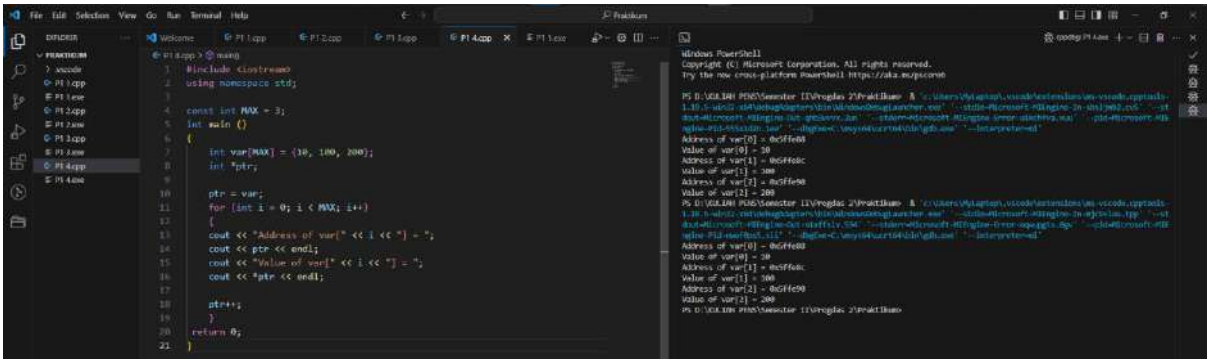
Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa kita dapat menginisialisasi pointer 'ptr' dengan 'NULL' lalu melihatnya dengan mencetak nilai pointer tersebut. Ketika pointer 'ptr' diinisialisasi dengan 'NULL' , yang menunjukkan bahwa pointer tidak menunjuk ke alamat memori apa pun. Maka, ketika dicetak hasil yang terlihat adalah '0' (nilai 'NULL').

4. Percobaan 4 – Pointer Arithmetic – Incrementing pointer

```
#include <iostream>
using namespace std;
 const int MAX =
3;
 int main
()
{     int var[MAX] = {10, 100,
200};     int *ptr;
    // let us have array address in pointer.
ptr = var;     for (int i = 0; i < MAX; i++)
    {         cout << "Address of var[" << i <<
"] = ";         cout << ptr << endl;
cout << "Value of var[" << i << "] = ";
cout << *ptr << endl;
        // point to the next location
ptr++;
    }
    return 0;
}
```

Hasil:



Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

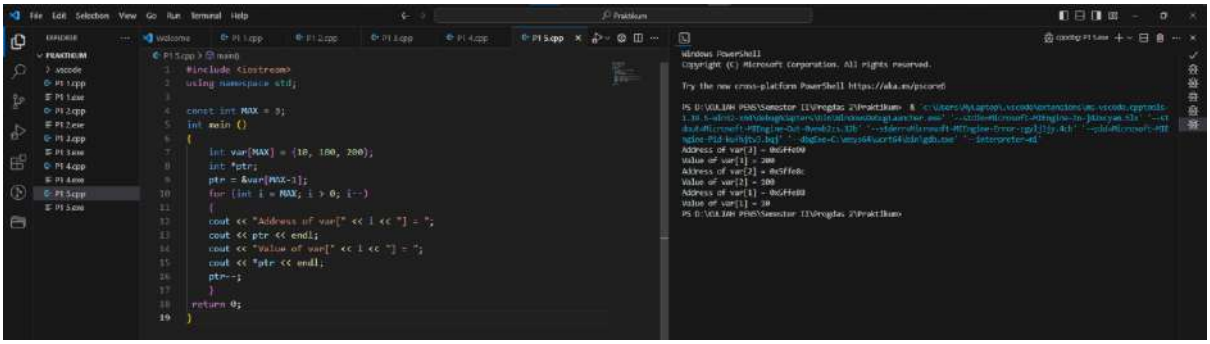
Program dalam percobaan ini dapat disimpulkan bahwa pointer bisa mengakses elemen-elemen array kemudian mencetak alamat serta nilai dari setiap elemen array. Dalam loop, pointer ptr digunakan untuk mengakses elemen-elemen array dengan menggunakan operator dereferensi *ptr. Alamat memori dari setiap elemen array tersebut dicetak menggunakan pointer ptr. Sehingga keluar nilai dari setiap elemen array yang dicetak dengan dereferensi pointer ptr. Setelah mencetak nilai dari suatu elemen, pointer ptr diinkremen untuk menunjuk ke elemen array berikutnya.

5. Percobaan 5 – Pointer Arithmetic – Decrementing pointer

```
#include <iostream>
using namespace std;
 const int MAX =
3;
```

```
 int main
()
{     int var[MAX] = {10, 100,
200};     int *ptr;
    // let us have address of the last element in
pointer.     ptr = &var[MAX-1];     for (int i = MAX; i >
0; i--)
    {         cout << "Address of var[" << i <<
"] = ";         cout << ptr << endl;
cout << "Value of var[" << i << "] = ";
cout << *ptr << endl;
        // point to the previous location
ptr--;
    }
    return 0;
}
```

Hasil:



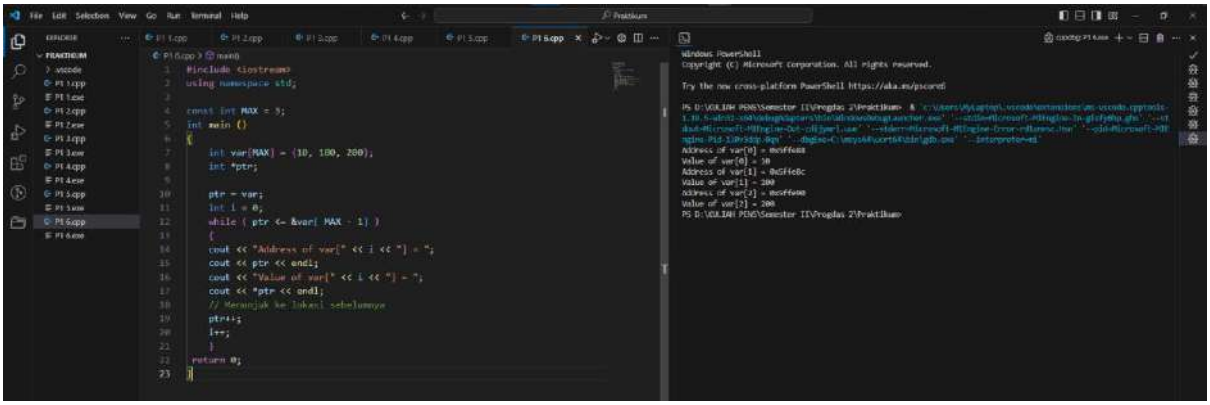
Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa penggunaan pointer dalam mengakses elemen-elemen array memungkinkan untuk mengakses secara langsung alamat memori dan nilai-nilai yang terkandung di dalamnya. Dengan menggunakan pointer, program dapat secara efisien mengakses dan memanipulasi elemen-elemen array dalam urutan yang berbeda, baik maju maupun mundur. Hal ini memberikan fleksibilitas dalam pengelolaan array dan memungkinkan untuk implementasi yang lebih dinamis dan efisien dalam pengembangan program.

6. Percobaan 6 – Membandingkan pointer

```
#include <iostream>
using namespace std;
 const int MAX =
3;
 int main
()
{
    int  var[MAX]  =  {10,  100,  200};
int *ptr;
    // let us have address of the first element in pointer.
    ptr = var;     int i = 0;
while ( ptr <= &var[ MAX - 1] )
    {         cout << "Address of var[" << i <<
"] = ";         cout << ptr << endl;
cout << "Value of var[" << i << "] = ";
cout << *ptr << endl;
        // point to the previous
location         ptr++;
i++;     }
    return 0;
}
```

Hasil:



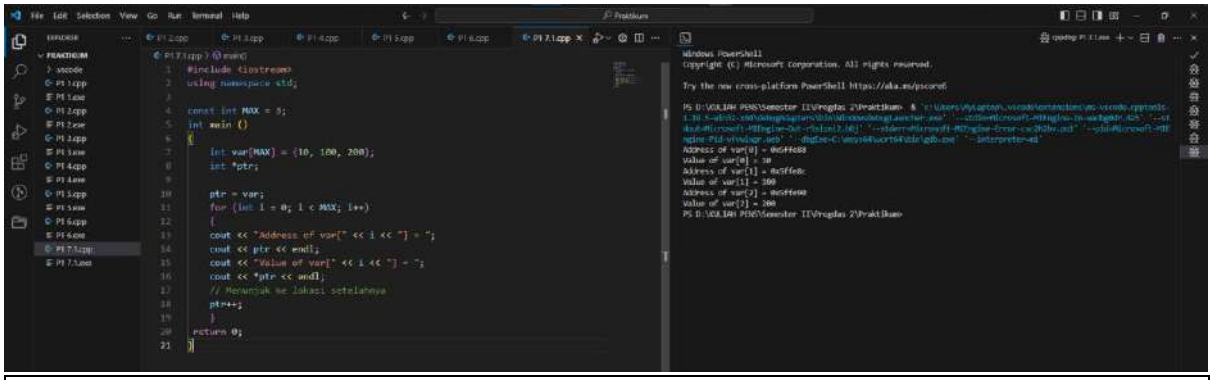
Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa penggunaan pointer dalam perulangan `while` untuk mengakses elemen-elemen array memberikan fleksibilitas dalam mengiterasi melalui array tanpa bergantung pada ukuran array yang spesifik. Dengan cara ini, program dapat dengan mudah beradaptasi dengan perubahan pada ukuran array atau bahkan array dinamis. Pointer juga digunakan untuk menunjuk ke alamat memori dari elemen pertama array, dan iterasi dilakukan hingga pointer mencapai alamat memori dari elemen terakhir array. Selama iterasi, program mencetak alamat dan nilai dari setiap elemen array yang ditunjuk oleh pointer, memberikan pemahaman yang jelas tentang struktur dan isi dari array.

7. Percobaan 7 – Pointer vs Array

```
#include <iostream>
using namespace std;
 const int MAX =
3;
 int main
()
{     int var[MAX] = {10, 100,
200};     int *ptr;
    // let us have array address in pointer.
ptr = var;     for (int i = 0; i < MAX; i++)
    {         cout << "Address of var[" << i <<
"] = ";         cout << ptr << endl;
cout << "Value of var[" << i << "] = ";
cout << *ptr << endl;
        // point to the next location
ptr++;
    }
    return 0;
}
```

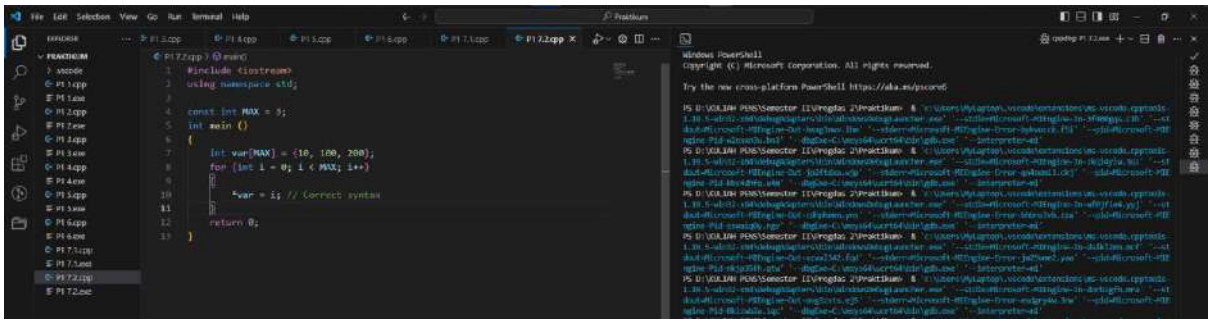
Hasil:



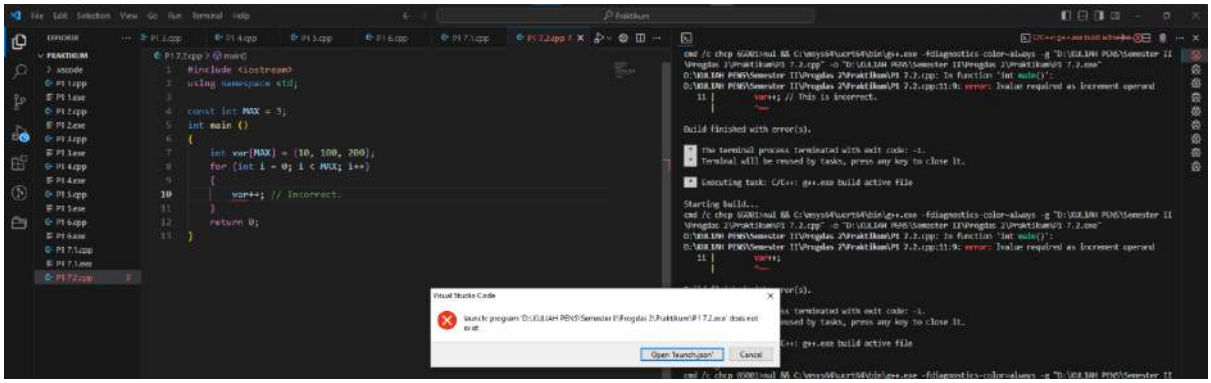
```
#include <iostream>
using namespace std;

const int MAX = 3;
 int main
()
{
    int var[MAX] = {10, 100, 200};
for (int i = 0; i < MAX; i++)
    {
        *var = i; // This is a correct syntax
var++; // This is incorrect.
    }
    return 0;
}
```

Hasil:

Dengan syntax yang benar



Dengan syntax yang salah



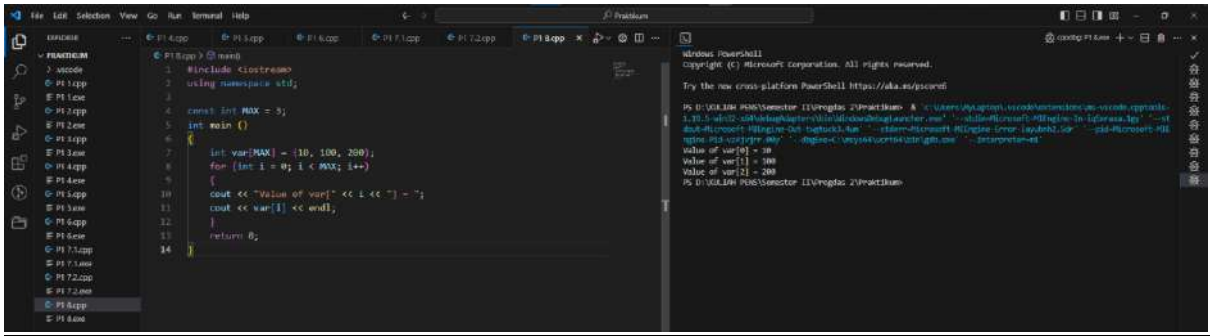
Dari dua percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa penggunaan pointer dengan benar memungkinkan akses yang tepat terhadap elemen-elemen array, yang dapat digunakan untuk iterasi dan manipulasi data dengan efisien. Pada percobaan pertama, penggunaan pointer dengan baik memungkinkan program untuk mencetak alamat dan nilai dari setiap elemen array secara berurutan. Namun, pada percobaan kedua, kesalahan dalam penggunaan pointer mengakibatkan program tidak dapat mencapai tujuannya, yaitu menginisialisasi nilai-nilai dalam array dengan urutan yang diinginkan.

8. Percobaan 8 – Array Pointer

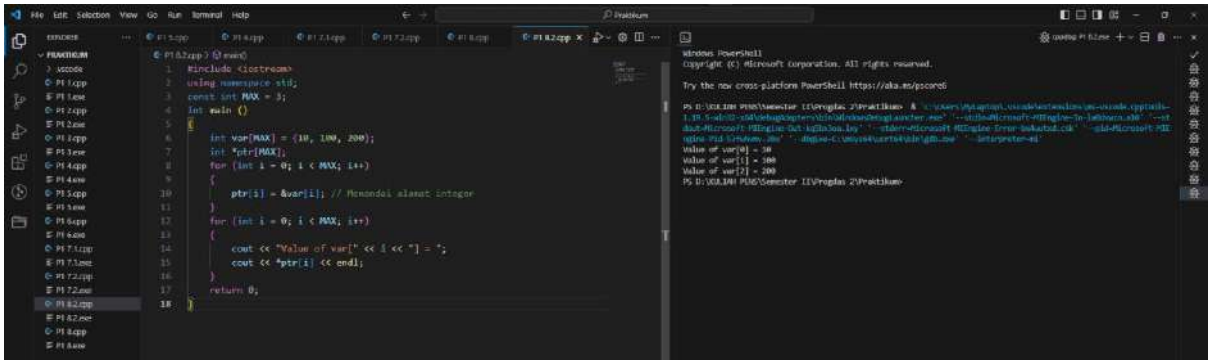
```
#include <iostream>
using namespace std;
 const int MAX =
3;
 int main
()
{     int var[MAX] = {10, 100,
200};     for (int i = 0; i <
MAX; i++)
    {         cout << "Value of var[" << i <<
"] = ";         cout << var[i] << endl;
    }
    return 0;
}
```

Hasil:



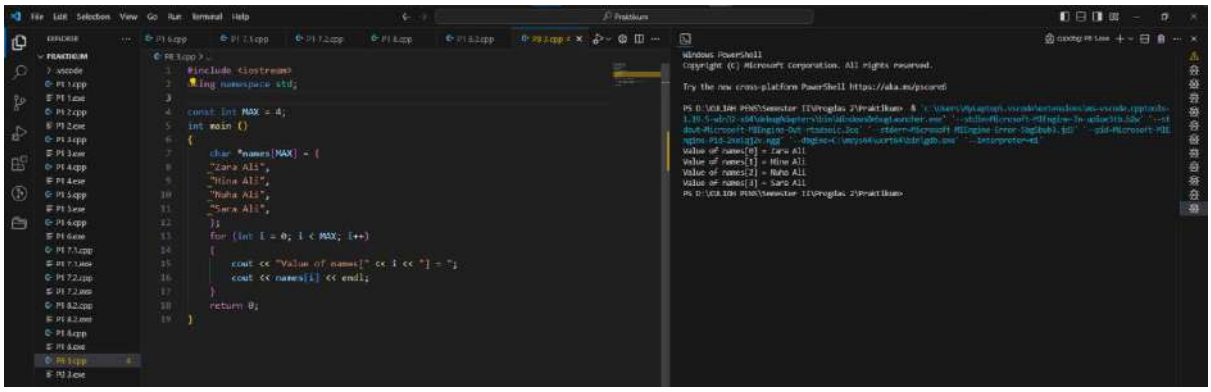
```
#include <iostream>
using namespace std;
 const int MAX =
3;
 int main
()
{     int var[MAX] = {10, 100,
200};     int *ptr[MAX];     for
(int i = 0; i < MAX; i++)
    {           ptr[i] = &var[i]; // assign the address of
integer.
    }     for (int i = 0; i <
MAX; i++)
    {
        cout << "Value of var[" << i << "] = ";
cout << *ptr[i] << endl;
    }
    return 0;
}
```

Hasil:



```
#include <iostream>
using namespace std;
 const int MAX =
4;
 int main
()
{     char *names[MAX] = {     "Zara Ali",
    "Hina Ali",
    "Nuha Ali",
    "Sara Ali",
    };     for (int i = 0; i <
MAX; i++)
    {         cout << "Value of names[" << i <<
"] = ";         cout << names[i] << endl;
    }
    return 0;
}
```

Hasil:



Dari tiga percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

1. Program pertama menggunakan array untuk menyimpan nilai-nilai integer dan kemudian mencetak nilai dari setiap elemen array. Dengan penggunaan array yang sederhana dan efektif untuk menyimpan data dalam urutan tertentu dan mengaksesnya secara langsung menggunakan indeks.

2. Program kedua menggunakan array dari pointer untuk menyimpan alamat memori dari elemen-elemen array integer. Setiap elemen pointer menunjuk ke alamat memori dari elemen yang sesuai dalam array `var`. Kemudian, program mengakses nilai dari setiap elemen array menggunakan pointer. Ini menunjukkan penggunaan array pointer untuk mengakses dan mengakses nilai-nilai dalam array dengan menggunakan pointer sebagai perantaranya.

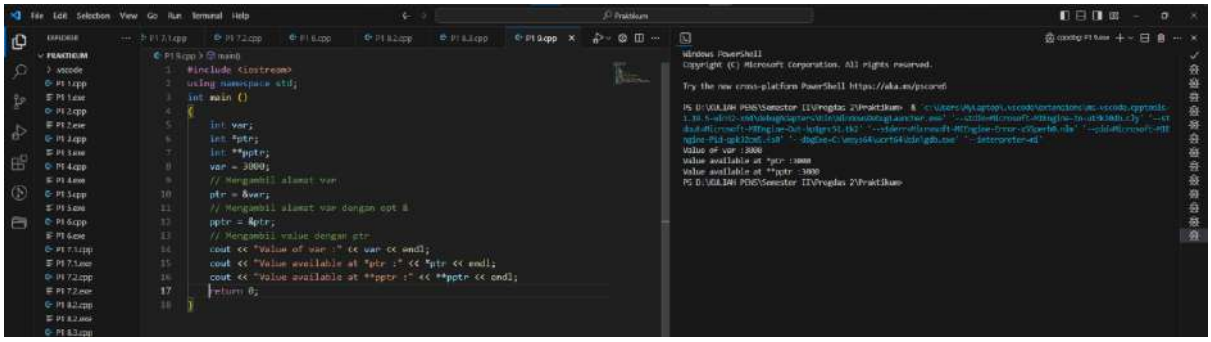
3. Program ketiga menggunakan array dari pointer untuk menyimpan alamat memori dari string. Setiap elemen pointer menunjuk ke string yang sesuai dalam array `names`. Kemudian, program mencetak nilai dari setiap elemen array pointer, yang merupakan alamat memori dari string. Ini menunjukkan penggunaan array pointer untuk menyimpan alamat memori dari tipe data yang berbeda, dalam hal ini, string.

Terdapat variasi penggunaan array dan pointer untuk menyimpan dan mengakses data dalam bentuk yang berbeda, baik itu nilai-nilai integer maupun string dari ketiga percobaan ini.

9. Percobaan 9 – Pointer – Pointer

```
#include <iostream>
using namespace std;
 int main
()
{     int var;
int *ptr;
int **pptr;
var = 3000;
    //  take  the  address  of  var
ptr = &var;
    // take the address of ptr using address of operator &
pptr = &ptr;
    // take the value using pptr     cout << "Value of
var :" << var << endl;     cout << "Value available at
*ptr :" << *ptr << endl;     cout << "Value available at
**pptr :" << **pptr << endl;     return 0;
}
```

Hasil:



Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa penggunaan pointer ganda memungkinkan untuk mengakses nilai dari sebuah variabel secara tidak langsung melalui

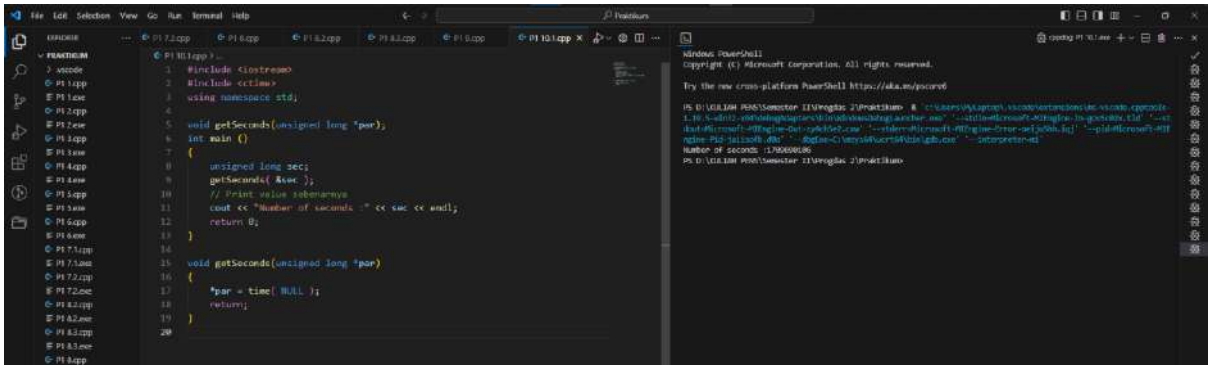
rangkaian pointer. Dalam contoh ini, variabel var diakses melalui pointer tunggal ptr, yang kemudian diakses lagi melalui pointer ganda pptr. Dengan menggunakan pointer ganda, kita dapat melihat bahwa program dapat mengakses nilai variabel var dengan menggunakan dereferensi ganda.

10. Percobaan 10 – Mempassing pointer ke fungsi

```
#include <iostream>
#include <ctime>
using namespace std;
 void getSeconds(unsigned long
*par);
```

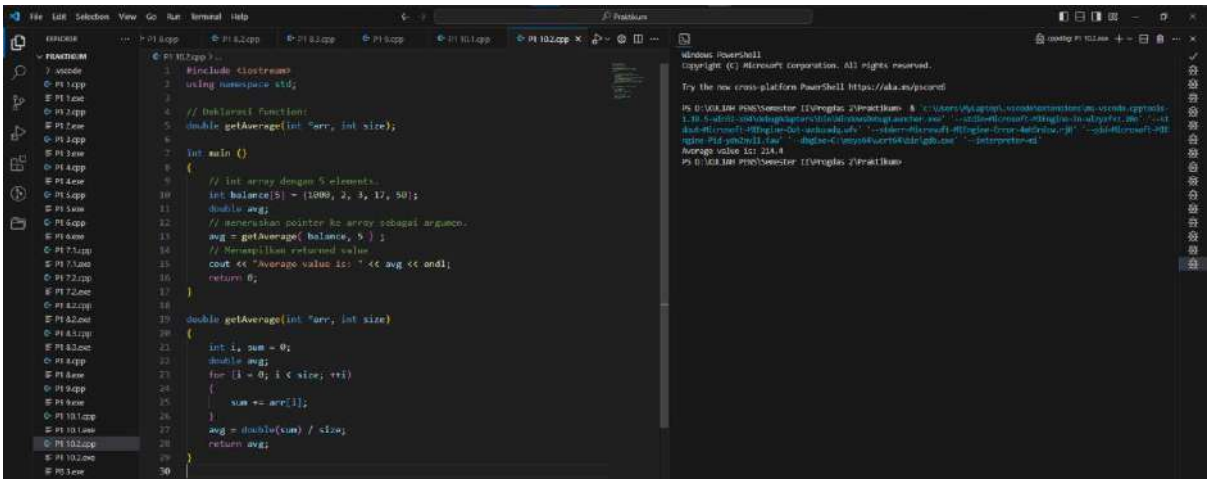
```
int main ()
{     unsigned long sec;     getSeconds( &sec );
// print the actual value     cout << "Number of
seconds :" << sec << endl;     return 0;
}  void getSeconds(unsigned long
*par)
{
    // get the current number of seconds
    *par = time( NULL );
return;
}
```

Hasil:



```
#include <iostream>
using namespace std;

// function declaration: double
getAverage(int *arr, int size);
 int main
()
{
    // an int array with 5 elements.
    int  balance[5]  =  {1000,  2,  3,  17,  50};
double avg;
    // pass pointer to the array as an
argument.     avg = getAverage( balance, 5 ) ;
// output the returned value
    cout << "Average value is: " << avg << endl;
return 0;
}
double getAverage(int *arr, int size)
{     int i, sum = 0;
double avg;     for (i = 0; i
< size; ++i)
    {
        sum += arr[i];
    }     avg = double(sum)
/ size;

    return avg;
}
```

Hasil:



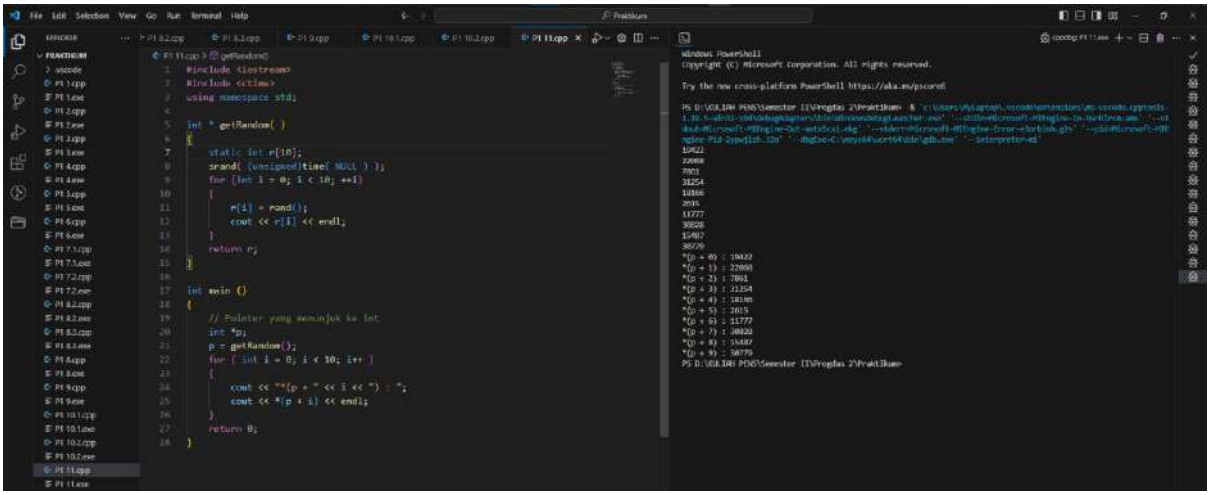
Dari dua percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Kedua program dalam percobaan ini dapat disimpulkan bahwa penggunaan pointer sebagai parameter dalam fungsi memungkinkan untuk mengakses dan memanipulasi nilai variabel yang berada di luar ruang lingkup fungsi tersebut. Dalam program pertama, pointer digunakan untuk mengambil nilai waktu dalam detik dan mengembalikannya ke dalam variabel di `main()`. Sedangkan dalam program kedua, pointer digunakan untuk meneruskan alamat memori dari array sebagai argumen fungsi, yang memungkinkan untuk melakukan operasi seperti menghitung rata-rata nilai dalam array tersebut. Dengan menggunakan pointer, kedua program tersebut menjadi lebih fleksibel dan dapat menangani data secara dinamis, tanpa perlu menyalin nilai-nilai variabel atau array.

11. Percobaan 11 – Mengembalikan pointer dari fungsi

```
#include <iostream>
#include <ctime>
using namespace std;

// function to generate and return random numbers.
int * getRandom( )
{     static int
r[10];

    // set the seed
srand( (unsigned)time( NULL ) );
for (int i = 0; i < 10; ++i)
    {
        r[i] = rand();
cout << r[i] << endl;
    }
    return r;
}

// main function to call above defined function. int
main ()
{
    // a pointer to an int.
int *p;     p = getRandom();
for ( int i = 0; i < 10; i++ )
    {
        cout << "*(p + " << i << ") : ";
cout << *(p + i) << endl;
    }
    return 0;
}
```

Hasil:



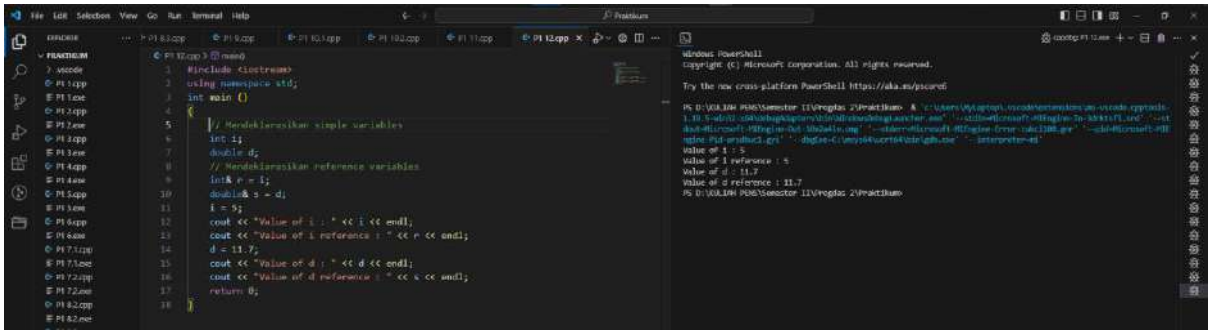
Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa penggunaan pointer dalam mengembalikan array secara efektif memungkinkan fungsi untuk menghasilkan dan mengembalikan array ke fungsi pemanggil. Fungsi `getRandom()` menghasilkan array yang berisi 10 angka acak dan mengembalikannya melalui pointer. Dengan menggunakan pointer, array tersebut dapat diakses dan ditampilkan di dalam fungsi `main()`. Penggunaan `static` dalam deklarasi array `r` memastikan bahwa array tersebut tetap ada di dalam memori bahkan setelah fungsi `getRandom()` selesai dieksekusi, sehingga pointer yang mengarah ke array tersebut masih valid. Dengan menggunakan pointer, program dapat secara efisien memanipulasi data tanpa perlu menyalin struktur data besar atau menghabiskan banyak memori.

12. Percobaan 12 – Referensi

```
#include <iostream>
using namespace std;
 int main
()
{
    // declare simple variables
int  i;  double  d;  //  declare
reference variables int& r = i;
double& s = d; i = 5; cout <<
"Value of i : " << i << endl;
cout << "Value of i reference :
" << r << endl; d = 11.7; cout <<
"Value of d : " << d << endl;
cout << "Value of d reference :
" << s << endl; return 0; }
```

Hasil:



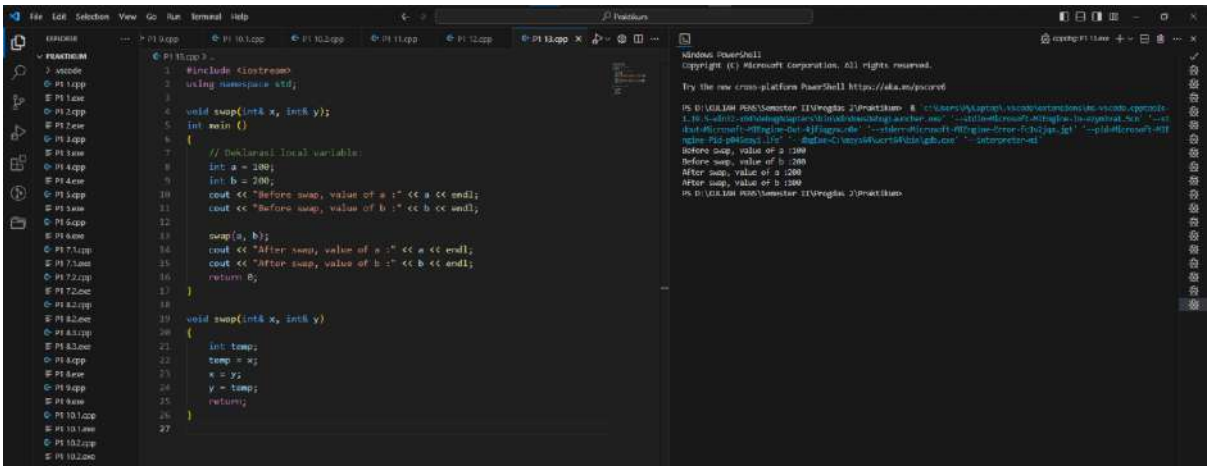
Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa penggunaan referensi memungkinkan untuk membuat alias dari variabel yang sudah ada, sehingga memungkinkan untuk mengakses dan memanipulasi variabel dengan cara yang lebih mudah dan intuitif. Dengan mendeklarasikan referensi, program dapat mengakses variabel asli dengan menggunakan nama referensi tersebut, tanpa perlu menggunakan operator dereferensi seperti pada penggunaan pointer. Hal ini membuat kode menjadi lebih mudah kita mengerti dan meminimalkan kesalahan yang mungkin terjadi saat bekerja dengan alamat memori.

13. Percobaan 13 – Referensi sebagai parameter

```
#include <iostream>
using namespace std;

// function declaration
void swap(int& x, int& y);
 int main
()
{
    // local variable declaration:
    int a = 100;     int b = 200;     cout <<
"Before swap, value of a :" << a << endl;     cout
<< "Before swap, value of b :" << b << endl;     /*
calling a function to swap the values.*/     swap(a,
b);     cout << "After swap, value of a :" << a <<
endl;     cout << "After swap, value of b :" << b <<
endl;     return 0;
}

// function definition to swap the values.
void swap(int& x, int& y)
{
    int temp;     temp = x; /* save the value
at address x */     x = y; /* put y into x */
y = temp; /* put x into y */     return;
}
```

Hasil:



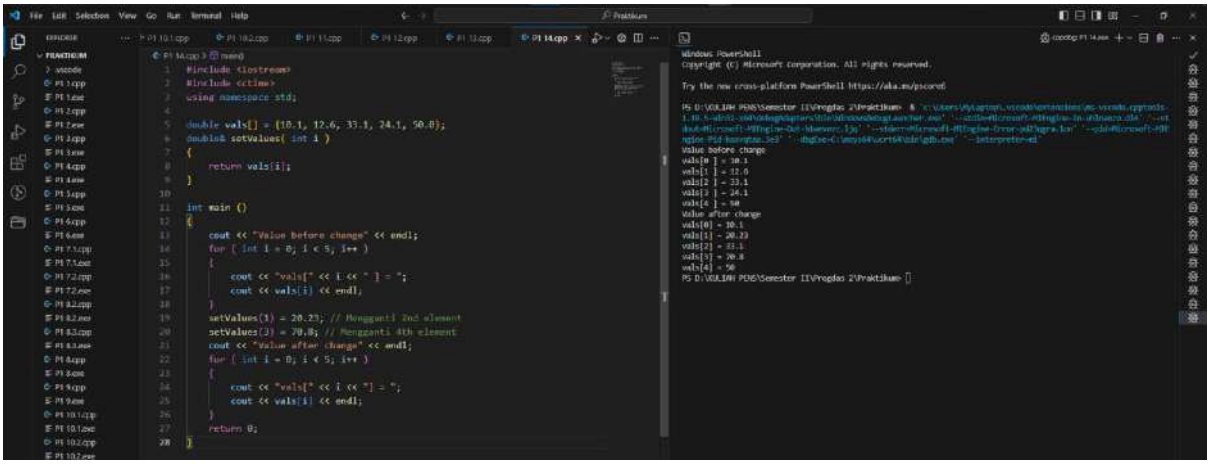
Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa penggunaan referensi dalam fungsi swap memungkinkan untuk menukar nilai antara dua variabel secara langsung, tanpa perlu menggunakan pointer atau nilai kembalian. Dengan menggunakan referensi sebagai parameter, fungsi swap dapat langsung mengakses dan memanipulasi nilai dari variabel yang dilewatkan, sehingga menghasilkan efek pertukaran nilai yang diinginkan. Sehingga kode menjadi lebih mudah dimengerti dan membantu dalam meningkatkan kejelasan dan keterbacaan kode.

14. Percobaan 14 – Referensi sebagai nilai pengembalian

```
#include <iostream>
#include <ctime>
using namespace std;
 double vals[] = {10.1, 12.6, 33.1, 24.1,
50.0};
 double& setValues( int
i )
{       return  vals[i];  //  return  a  reference  to  the  ith
element }

// main function to call above defined function. int
main ()
{     cout << "Value before change" <<
endl;     for ( int i = 0; i < 5; i++ )
    {         cout << "vals[" << i <<
" ] = ";         cout << vals[i] <<
endl;
    }     setValues(1) = 20.23; // change 2nd
element     setValues(3) = 70.8; // change 4th
element     cout << "Value after change" <<
endl;     for ( int i = 0; i < 5; i++ )
    {         cout << "vals[" << i <<
"] = ";         cout << vals[i] <<
endl;
    }
    return 0;
}
```

Hasil:



Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Program dalam percobaan ini dapat disimpulkan bahwa penggunaan referensi sebagai nilai kembalian dari fungsi setValues memungkinkan untuk mengakses dan memanipulasi nilai dari array vals secara langsung melalui referensi yang dikembalikan. Dengan cara ini, program dapat mengubah nilai dari elemen-elemen array vals dengan menggunakan fungsi setValues sebagai perantara, tanpa perlu mengakses array secara langsung. Hasilnya ialah terdapat fleksibilitas dan kemudahan dalam memanipulasi data dalam array.